

TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN

-----□-----



HỌC PHẦN: THỰC TẬP ĐỒ ÁN CƠ SỞ NGÀNH
HỌC KỲ I, NĂM HỌC 2025-2026

XÂY DỰNG ỨNG DỤNG MÔ PHỎNG GIẢI THUẬT TÌM KIẾM TRÊN ĐỒ THỊ

Giáo viên hướng dẫn:

ThS. Trầm Hoàng Nam

Sinh viên thực hiện:

Họ tên: Cô Nhân Quý
MSSV: 110122150
Lớp DA22TTB

Vĩnh Long, tháng 12 năm 2025

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Vĩnh Long, ngày tháng năm

Giảng viên hướng dẫn

(Ký tên và ghi rõ họ tên)

NHẬN XÉT CỦA THÀNH VIÊN HỘI ĐỒNG

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Vĩnh Long, ngày tháng năm

Thành viên hội đồng

(Ký tên và ghi rõ họ tên)

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến ThS. Trần Hoàng Nam, giáo viên hướng dẫn, người đã tận tình hướng dẫn, chỉ bảo và tạo điều kiện thuận lợi để em trong suốt quá trình thực hiện Thực tập Đồ án cơ sở ngành với đề tài " Xây dựng ứng dụng mô phỏng giải thuật tìm kiếm trên đồ thị ". Với những kiến thức quý báu của thầy đã giúp em vượt qua những khó khăn trong quá trình nghiên cứu và hoàn thiện báo cáo.

Sự nhiệt tình và kiên nhẫn của thầy đã giúp em hoàn thành đồ án một cách suôn sẻ và hiệu quả hơn. Em thật sự rất biết ơn và mong rằng sẽ tiếp tục nhận được sự giúp đỡ từ thầy trong những lần sau.

Xin chân thành cảm ơn thầy!

Sinh viên ký và ghi rõ họ và tên

Sinh viên thực hiện

Cô Nhân Quý

MỤC LỤC

TÓM TẮT ĐỒ ÁN	9
MỞ ĐẦU	10
CHƯƠNG 1: TỔNG QUAN	13
1.1. Tổng quan về Đồ thị trong Khoa học máy tính.....	13
1.2. Các loại giải thuật tìm kiếm cơ bản trên đồ thị	13
1.3. Vai trò và Ý nghĩa thực tiễn	13
1.4. Quy trình nghiên cứu và phát triển ứng dụng:	14
CHƯƠNG 2: NGHIÊN CỨU LÝ THUYẾT.....	15
2.1. Cơ sở lý thuyết về Đồ thị.....	15
2.2. Giải thuật Breadth-First Search (BFS).....	15
2.3. Giải thuật Depth-First Search (DFS).....	16
2.4. So sánh giải thuật BFS và DFS	17
2.5. Ý nghĩa của việc mô phỏng trực quan.....	17
CHƯƠNG 3: HIỆN THỰC HÓA NGHIÊN CỨU	18
3.1. Công cụ và môi trường phát triển.....	18
3.2. Thiết kế giao diện và cấu trúc dữ liệu	20
3.2.1. Thiết kế giao diện	20
3.2.2. Cấu trúc dữ liệu:	20
3.3. Hiện thực hóa giải thuật	21
3.3.1. Giải thuật Breadth-First Search (BFS).....	21
3.3.2. Giải thuật Depth-First Search (DFS).....	21
3.3.3. So sánh song song thuật toán BFS và DFS	22
3.4. Mô phỏng trực quan và Tương tác	22
3.4.1. Cơ chế Chuyển động (Animation)	22

3.4.2. Hiệu ứng màu sắc	22
3.4.3. Tương tác người dùng	23
CHƯƠNG 4: KẾT QUẢ NGHIÊN CỨU	24
4.1 Tổng quan kết quả đạt được	24
4.2. Kết quả mô phỏng	24
4.2.1. Giao diện ứng dụng	24
4.2.2. Mô phỏng thuật toán BFS/DFS	25
4.2.3. Thử nghiệm tương tác	29
4.2.4. Chức năng duyệt song song.....	30
4.3. Nhận xét chung.....	30
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	32
5.1. Kết quả đạt được.....	32
5.2. Ưu nhược điểm.....	32
5.3. Hướng phát triển.....	32
5.4. Kết luận.....	33
TÀI LIỆU THAM KHẢO	34

DANH MỤC BẢNG BIỂU

Bảng 1: Bảng so sánh giải thuật BFS và DFS17

DANH MỤC HÌNH ẢNH

Hình 1 Ngôn ngữ HTML, CSS và JavaScript	18
Hình 2 NodeJs.....	18
Hình 3 Nền tảng phát triển ứng dụng ElectronJS	19
Hình 4 Phần mềm Visual Studio Code	19
Hình 5 Công cụ hỗ trợ quản lý mã nguồn Git và GitHub.....	19
Hình 6 Giao diện chính.....	25
Hình 8 Kết quả mô phỏng thuật toán BFS.....	27
Hình 9 Kết quả mô phỏng thuật toán DFS	28
Hình 10 Kết quả thay đổi vị trí chướng ngại vật	29
Hình 11 Mô phỏng chạy BFS sau khi thay đổi vị trí chướng ngại vật.....	29
Hình 12 Chức năng duyệt song song	30

TÓM TẮT ĐỒ ÁN

Trong lĩnh vực khoa học máy tính, các giải thuật tìm kiếm trên đồ thị như Breadth-First Search (BFS) và Depth-First Search (DFS) đóng vai trò nền tảng cơ sở. Tuy nhiên, nếu chỉ tiếp cận qua lý thuyết, sinh viên thường khó hình dung được cách thuật toán thực sự vận hành: thứ tự duyệt các đỉnh, cách mở rộng tìm kiếm và quá trình hình thành đường đi.

Đồ án “Xây dựng ứng dụng mô phỏng giải thuật tìm kiếm trên đồ thị” hướng đến việc tạo ra một công cụ trực quan hóa sinh động trên nền tảng Web (HTML/CSS/JavaScript). Ứng dụng cho phép người dùng tương tác trực tiếp: vẽ tường, đặt điểm bắt đầu/kết thúc, nhập dữ liệu từ ma trận số và quan sát sự khác biệt giữa BFS và DFS theo thời gian thực.

Kết quả đạt được:

Ứng dụng đã được xây dựng thành công với giao diện thân thiện, trực quan, hỗ trợ so sánh song song hiệu suất của BFS và DFS. Qua đó, người học dễ dàng nắm bắt bản chất của từng giải thuật, thay vì chỉ dựa vào lý thuyết khô khan.

MỞ ĐẦU

Lý do chọn đề tài

Trong quá trình học tập ngành Công nghệ thông tin, sinh viên thường phải tiếp cận nhiều thuật toán nền tảng. Các giải thuật tìm kiếm trên đồ thị như BFS và DFS không chỉ xuất hiện trong môn Lý thuyết đồ thị mà còn là nền tảng cho nhiều ứng dụng thực tế: từ việc tìm đường đi trong hệ thống bản đồ, giải quyết bài toán mê cung, cho đến các ứng dụng trong trí tuệ nhân tạo.

Tuy nhiên, việc học các thuật toán này chủ yếu thông qua lý thuyết và ví dụ trên giáo trình khiến sinh viên khó hình dung được cách chúng hoạt động từng bước. Điều này dẫn đến việc nắm bắt kiến thức chưa thật sự sâu, khó áp dụng vào các bài toán thực tế.

Chính vì vậy, đề án thực hiện mong muốn xây dựng một công cụ mô phỏng trực quan, cho phép người dùng quan sát quá trình duyệt của BFS và DFS. Ứng dụng không chỉ giúp sinh viên dễ dàng tiếp cận kiến thức mà còn tạo hứng thú học tập, biến những khái niệm trừu tượng thành trải nghiệm sinh động.

Mục tiêu đề tài

Mục tiêu tổng quát:

Xây dựng một ứng dụng mô phỏng trực quan các giải thuật tìm kiếm trên đồ thị (BFS và DFS), giúp sinh viên dễ dàng quan sát, hiểu rõ cơ chế hoạt động và so sánh sự khác biệt giữa hai thuật toán.

Mục tiêu cụ thể:

- Thiết kế giao diện đồ họa thân thiện, cho phép người dùng tương tác trực tiếp với lưới (vẽ tường, đặt điểm bắt đầu/kết thúc).
- Hiện thực hóa giải thuật BFS và DFS bằng JavaScript, mô phỏng quá trình duyệt từng bước với hiệu ứng màu sắc.
- Hỗ trợ nhập dữ liệu từ ma trận số để mô phỏng các bài toán thực tế.
- Cung cấp chức năng so sánh song song BFS và DFS để người học thấy rõ ưu/nhược điểm.

- Đảm bảo ứng dụng chạy ổn định, dễ sử dụng, phục vụ cho việc học tập và nghiên cứu cơ sở ngành.

Đối tượng và dữ liệu nghiên cứu

Đối tượng nghiên cứu

- Các giải thuật tìm kiếm cơ bản trên đồ thị: BFS (Breadth-First Search) và DFS (Depth-First Search).
- Cách biểu diễn đồ thị bằng ma trận kề hoặc danh sách kề.
- Sinh viên ngành Công nghệ thông tin, đặc biệt là những người đang học môn Lý thuyết đồ thị hoặc các môn liên quan.

Phương pháp thực hiện

Phương pháp lý thuyết:

- Nghiên cứu cơ sở lý thuyết về đồ thị: tìm hiểu khái niệm đỉnh, cạnh, cách biểu diễn bằng ma trận kề và danh sách kề.
- Phân tích giải thuật BFS và DFS: nghiên cứu nguyên lý hoạt động, cấu trúc dữ liệu hỗ trợ (Queue cho BFS, Stack cho DFS).
- So sánh đặc điểm: chỉ ra sự khác biệt về cách duyệt, phạm vi tìm kiếm và ưu nhược điểm của từng thuật toán.
- Xây dựng mô hình lý thuyết: thiết kế sơ đồ khối, quy trình thuật toán để làm cơ sở cho việc lập trình mô phỏng.

Ưu điểm:

- Giúp người thực hiện nắm chắc kiến thức nền tảng, tránh sai sót khi lập trình.
- Có cơ sở khoa học rõ ràng, dễ dàng giải thích và trình bày trong báo cáo.
- Tạo nền móng để mở rộng sang các thuật toán nâng cao khác.

Hạn chế:

- Mang tính lý thuyết, chưa phản ánh hết các vấn đề phát sinh khi triển khai thực tế.
- Khó tạo hứng thú cho người học nếu chỉ dừng lại ở phân tích giấy tờ.

Phương pháp thực nghiệm:

- Thiết kế giao diện: dùng HTML/CSS để xây dựng lưới trực quan, cho phép người dùng tương tác.
- Hiện thực thuật toán: lập trình BFS và DFS bằng JavaScript, kết hợp Electron để triển khai ứng dụng desktop.
- Mô phỏng trực quan: sử dụng màu sắc và hiệu ứng để minh họa trạng thái các ô (đã duyệt, đường đi cuối cùng).
- Kiểm thử và đánh giá: chạy thử với nhiều dữ liệu, nhập ma trận số, so sánh song song BFS và DFS.
- Hoàn thiện sản phẩm: tối ưu tốc độ hiển thị, sửa lỗi, bổ sung chức năng cần thiết và viết báo cáo tổng kết.

Ưu điểm:

- Trực quan, sinh động, dễ tạo hứng thú cho người học.
- Cho phép kiểm chứng lý thuyết bằng thực nghiệm, giúp hiểu sâu hơn.
- Có thể mở rộng, nâng cấp thêm nhiều thuật toán khác.
- Tạo ra sản phẩm cụ thể, có giá trị ứng dụng trong giảng dạy.

Hạn chế:

- Phụ thuộc vào kỹ năng lập trình và công cụ (HTML/CSS/JS/Electron).
- Hiệu năng có thể giảm khi mô phỏng trên lưới lớn.
- Giao diện còn đơn giản, chưa tích hợp nhiều tính năng nâng cao.
- Chủ yếu phục vụ mục đích học tập, chưa tối ưu cho ứng dụng thực tế quy mô lớn.

CHƯƠNG 1: TỔNG QUAN

1.1. Tổng quan về Đồ thị trong Khoa học máy tính

Khái niệm

Đồ thị là một cấu trúc dữ liệu quan trọng trong khoa học máy tính, được sử dụng để mô tả mối quan hệ giữa các đối tượng. Một đồ thị gồm tập hợp các đỉnh (vertices) và cạnh (edges) nối giữa các đỉnh [1]. Đồ thị có thể biểu diễn nhiều bài toán thực tế như: mạng xã hội, hệ thống giao thông, sơ đồ mạng máy tính, hay bài toán mê cung.

Trong lĩnh vực thuật toán, đồ thị là nền tảng để nghiên cứu các bài toán tìm kiếm, tối ưu hóa và phân tích dữ liệu. Việc hiểu rõ cách biểu diễn và xử lý đồ thị là bước khởi đầu quan trọng cho các môn học nâng cao như Trí tuệ nhân tạo (AI), Khoa học dữ liệu (Data Science) và Phân tích mạng (Network Analysis).

1.2. Các loại giải thuật tìm kiếm cơ bản trên đồ thị

Trong đồ án này, chúng ta tập trung vào hai loại giải thuật tìm kiếm phổ biến nhất:

- **Breadth-First Search (BFS):** tìm kiếm theo chiều rộng, sử dụng cấu trúc dữ liệu hàng đợi (Queue). BFS duyệt các đỉnh theo từng lớp, đảm bảo tìm được đường đi ngắn nhất trong đồ thị không trọng số [2].
- **Depth-First Search (DFS):** tìm kiếm theo chiều sâu, sử dụng cấu trúc dữ liệu ngăn xếp (Stack) hoặc đệ quy. DFS đi sâu vào một nhánh trước khi quay lại, phù hợp cho việc kiểm tra tính liên thông hoặc phát hiện chu trình.

Cả BFS và DFS đều là nền tảng để phát triển các thuật toán nâng cao hơn như Dijkstra, A*, hay Bellman-Ford.

1.3. Vai trò và Ý nghĩa thực tiễn

- **Trong học tập:** giúp sinh viên hình dung rõ ràng cách thuật toán hoạt động, thay vì chỉ dựa vào lý thuyết.
- **Trong thực tế:** được ứng dụng trong tìm đường đi ngắn nhất, giải quyết bài toán mê cung, phân tích mạng xã hội, hệ thống khuyến nghị.
- **Trong nghiên cứu:** là nền tảng để phát triển các thuật toán phức tạp hơn, phục vụ cho AI và khoa học dữ liệu.

1.4. Quy trình nghiên cứu và phát triển ứng dụng:

Đồ án tuân theo quy trình chuẩn gồm các bước sau:

1. Xác định vấn đề: nhu cầu trực quan hóa BFS và DFS để hỗ trợ học tập.
2. Nghiên cứu lý thuyết: tìm hiểu cơ sở đồ thị, nguyên lý BFS/DFS, ưu nhược điểm.
3. Thiết kế và hiện thực hóa: xây dựng giao diện bằng HTML/CSS, lập trình thuật toán bằng JavaScript, kết hợp Electron để tạo ứng dụng desktop.
4. Mô phỏng và trực quan hóa: sử dụng màu sắc, hiệu ứng để minh họa trạng thái duyệt.
5. Kiểm thử và đánh giá: chạy thử với nhiều dữ liệu, so sánh song song BFS và DFS.
6. Hoàn thiện và báo cáo: tối ưu tốc độ hiển thị, bổ sung chức năng nhập ma trận, viết báo cáo tổng kết.

CHƯƠNG 2: NGHIÊN CỨU LÝ THUYẾT

2.1. Cơ sở lý thuyết về Đồ thị

Đồ thị là một cấu trúc dữ liệu bao gồm tập hợp các đỉnh (vertices) và cạnh (edges).

Đồ thị có thể phân loại thành:

- Đồ thị vô hướng: cạnh không có hướng, biểu diễn mối quan hệ hai chiều.
- Đồ thị có hướng: cạnh có hướng, biểu diễn mối quan hệ một chiều.
- Đồ thị có trọng số: cạnh gắn với một giá trị (trọng số), thường dùng để biểu diễn chi phí hoặc khoảng cách.

Các cách biểu diễn đồ thị phổ biến:

- Ma trận kề (Adjacency Matrix): dùng ma trận 2 chiều để lưu thông tin cạnh.
- Danh sách kề (Adjacency List): mỗi đỉnh lưu danh sách các đỉnh kề.

2.2. Giải thuật Breadth-First Search (BFS)

- **Khái niệm:**

Trong lý thuyết đồ thị, tìm kiếm theo chiều rộng (Breadth-First Search – BFS) là một thuật toán duyệt đồ thị, trong đó việc tìm kiếm bắt đầu từ một đỉnh nguồn và lần lượt mở rộng sang các đỉnh kề.

- **Nguyên lý hoạt động:**

BFS duyệt đồ thị theo từng lớp, bắt đầu từ đỉnh xuất phát, lần lượt duyệt các đỉnh kề trước khi đi sâu hơn, đảm bảo mỗi đỉnh được duyệt theo khoảng cách ngắn nhất từ nguồn.

- **Cấu trúc dữ liệu sử dụng:** Hàng đợi (Queue)

Cách hoạt động:

1. Đỉnh nguồn được đưa vào hàng đợi.
2. Lấy đỉnh ở đầu hàng đợi ra, duyệt các láng giềng chưa thăm.
3. Các láng giềng này được đưa vào cuối hàng đợi.

4. Lặp lại cho đến khi hàng đợi rỗng hoặc gặp đích.

- **Đặc điểm:**

- Luôn tìm được đường đi ngắn nhất trong đồ thị không trọng số.
- Thứ tự duyệt rõ ràng, dễ quan sát.

- **Ứng dụng:**

Tìm đường đi ngắn nhất, giải bài toán mê cung, kiểm tra tính liên thông.

2.3. Giải thuật Depth-First Search (DFS)

- **Khái niệm:**

Tìm kiếm theo chiều sâu (Depth-First Search – DFS) là một thuật toán duyệt hoặc tìm kiếm trên cây hoặc đồ thị. Thuật toán khởi đầu tại gốc (hoặc một đỉnh bất kỳ) và phát triển xa nhất có thể theo mỗi nhánh trước khi quay lại.

- **Nguyên lý hoạt động:**

DFS đi sâu vào một nhánh cho đến khi không thể đi tiếp, sau đó quay lui để duyệt nhánh khác.

- **Cấu trúc dữ liệu sử dụng:** Ngăn xếp (Stack) hoặc đệ quy.

1. Đỉnh nguồn được đưa vào ngăn xếp.
2. Lấy đỉnh ở đầu ngăn xếp ra, duyệt các láng giềng chưa thăm.
3. Các láng giềng này được đưa vào ngăn xếp, và sẽ được xử lý trước các đỉnh thêm vào trước đó.
4. Khi không còn láng giềng, thuật toán “quay lui” bằng cách lấy phần tử tiếp theo trong ngăn xếp.

- **Đặc điểm:**

- Không đảm bảo tìm được đường đi ngắn nhất.
- Thích hợp để kiểm tra chu trình, phân tích cấu trúc đồ thị.

- **Ứng dụng:**

Phát hiện chu trình, phân tích mạng, duyệt toàn bộ đồ thị.

2.4. So sánh giải thuật BFS và DFS

Tiêu chí	BFS	DFS
Cấu trúc dữ liệu	Queue	Stack / Đệ quy
Thứ tự duyệt	Theo lớp (lan rộng)	Theo nhánh (đi sâu)
Đường đi ngắn nhất	Có (trong đồ thị không trọng số)	Không đảm bảo
Ứng dụng	Tìm đường đi ngắn nhất, mê cung	Phát hiện chu trình, phân tích cấu trúc
Trực quan mô phỏng	Dễ quan sát, lan tỏa từng lớp	Thể hiện rõ quá trình quay lui

Bảng 1: Bảng so sánh giải thuật BFS và DFS

2.5. Ý nghĩa của việc mô phỏng trực quan

- Giúp sinh viên dễ dàng hình dung cách thuật toán hoạt động từng bước.
- Tạo hứng thú học tập nhờ hiệu ứng màu sắc và sự tương tác trực tiếp.
- Làm rõ sự khác biệt giữa BFS và DFS, từ đó hiểu sâu hơn về ưu nhược điểm của từng thuật toán.
- Là nền tảng để mở rộng nghiên cứu sang các thuật toán nâng cao như Dijkstra, A*.

CHƯƠNG 3: HIỆN THỰC HÓA NGHIÊN CỨU

3.1. Công cụ và môi trường phát triển

- **Ngôn ngữ lập trình:**

HTML, CSS và JavaScript. Đây là bộ 3 công nghệ phổ biến nhất hiện nay cho việc xây dựng cấu trúc trang, xử lý logic thuật toán [3] và định dạng giao diện và màu sắc (cho các trạng thái đối với đồ án).



Hình 1 Ngôn ngữ HTML, CSS và JavaScript

- **Môi trường thực thi**

Node.js: Đóng vai trò là môi trường chạy mã JavaScript phía máy khách (Client-side) và quản lý các gói thư viện thông qua NPM (Node Package Manager) [5]. Node.js giúp ứng dụng có khả năng tương tác với hệ điều hành (đọc/ghi file, quản lý cửa sổ) mà trình duyệt web thông thường không làm được.



Hình 2 NodeJs

- **Thư viện sử dụng**

ElectronJS: Thư viện mã nguồn mở dựa trên Node.js và Chromium [4]. Electron giúp đóng gói toàn bộ mã nguồn HTML/CSS/JS cho phép chạy độc lập trên nhiều hệ điều hành.



Hình 3 Nền tảng phát triển ứng dụng ElectronJS

- **Nền tảng thực nghiệm:**

- Visual Studio Code (VS Code): Môi trường phát triển tích hợp (IDE) dùng để tổ chức cấu trúc dự án và chỉnh sửa mã nguồn.



Hình 4 Phần mềm Visual Studio Code

- **Quản lý mã nguồn:** Git và GitHub để lưu trữ và quản lý phiên bản mã nguồn.



Hình 5 Công cụ hỗ trợ quản lý mã nguồn Git và GitHub

3.2. Thiết kế giao diện và cấu trúc dữ liệu

3.2.1. Thiết kế giao diện

Giao diện ứng dụng được chia thành hai khu vực chính dựa trên file index.html và style.css:

- **Bảng điều khiển (Control Panel - Bên trái):** Chứa các nút chức năng điều khiển quy trình mô phỏng:
 - Nhóm lệnh thuật toán: Chạy BFS, Chạy DFS, So sánh song song.
 - Nhóm lệnh thao tác: Reset lưới (Xóa / Tạo Mới).
 - Khu vực nhập liệu: Textarea cho phép người dùng nhập ma trận số (0/1) để tạo bản đồ nhanh.
 - Bảng chú thích: Giải thích ý nghĩa màu sắc của các ô.
- **Khu vực hiển thị lưới (Grid Container - Bên phải):**
 - Là nơi hiển thị đồ thị dưới dạng lưới ô vuông 20x20 (mặc định).
 - Mỗi ô vuông là một thẻ <div> có ID định danh dạng grid-row-col (ví dụ: grid-5-5), giúp dễ dàng truy xuất và thay đổi màu sắc qua CSS class.

3.2.2. Cấu trúc dữ liệu:

Dựa trên mã nguồn script.js, hệ thống sử dụng các cấu trúc dữ liệu sau để quản lý trạng thái:

- **Tọa độ đỉnh (Node):** Mỗi đỉnh trên đồ thị được định nghĩa bằng một object { r: row_index, c: col_index }.
- **Biến trạng thái toàn cục:**
 - startNode: Lưu tọa độ điểm bắt đầu (Mặc định: {r: 5, c: 5}).
 - endNode: Lưu tọa độ điểm đích (Mặc định: {r: 15, c: 15}).
 - isRunning: Cờ hiệu (Boolean) để khóa các thao tác người dùng khi thuật toán đang chạy.
- **Quản lý Tường (Walls):**
 - Sử dụng cấu trúc Set trong JavaScript để lưu trữ vị trí các bức tường.

- Dữ liệu được lưu dưới dạng chuỗi string "r-c". Việc dùng Set giúp thao tác kiểm tra một ô có phải là tường hay không (walls.has()) đạt độ phức tạp $O(1)$, tối ưu hơn so với duyệt mảng.
- **Hàng đợi và Ngăn xếp (Queue & Stack):**
 - Sử dụng mảng (Array) của JavaScript để cài đặt Queue (cho BFS - cơ chế FIFO) và Stack (cho DFS - cơ chế LIFO).
- **Mảng truy vết (Traceback):**
 - Sử dụng mảng 2 chiều parent[r][c] để lưu tọa độ của đỉnh cha (đỉnh liền trước nó). Cấu trúc này phục vụ cho việc truy vết ngược từ điểm đích về điểm đầu để vẽ đường đi ngắn nhất.

3.3. Hiện thực hóa giải thuật

Quy trình xử lý thuật toán được chia thành 3 bước: Khởi tạo, Duyệt đồ thị và Truy vết đường đi.

3.3.1. Giải thuật Breadth-First Search (BFS)

Nguyên lý: Duyệt đều ra xung quanh theo từng lớp.

- Khởi tạo Queue và đưa startNode vào.
- Lặp khi Queue không rỗng:
 - Lấy đỉnh current ra khỏi đầu hàng đợi (shift).
 - Nếu current là endNode: Dừng và truy vết.
 - Xét 4 ô lân cận (Lên, Xuống, Trái, Phải). Nếu ô lân cận hợp lệ (không ra ngoài lưới, không phải tường, chưa duyệt):
 - + Đánh dấu đã duyệt.
 - + Lưu parent của ô lân cận là current.
 - + Đẩy ô lân cận vào Queue.

3.3.2. Giải thuật Depth-First Search (DFS)

Nguyên lý: Duyệt sâu theo một nhánh đến khi không đi được nữa thì quay lui (Backtracking).

- Khởi tạo Stack và đưa startNode vào.
- Lặp khi Stack không rỗng:
 - + Lấy đỉnh current ra khỏi đỉnh ngăn xếp (pop).
 - + Các bước kiểm tra đích và xét lân cận tương tự BFS, nhưng thứ tự thêm vào Stack sẽ quyết định hướng đi ưu tiên (ví dụ: ưu tiên đi xuống trước).

3.3.3. So sánh song song thuật toán BFS và DFS

- Hệ thống khởi tạo hai lưới DOM riêng biệt (grid1 và grid2).
- Sử dụng Promise.all hoặc gọi hai hàm async đồng thời để chạy BFS trên grid1 và DFS trên grid2 cùng lúc.
- Giúp người dùng so sánh trực quan tốc độ lan truyền của hai thuật toán (BFS loang rộng chậm hơn nhưng tìm đường tối ưu, DFS đi nhanh nhưng đường đi có thể không tối ưu).

3.4. Mô phỏng trực quan và Tương tác

Để ứng dụng trở nên sinh động, đề tài áp dụng các kỹ thuật xử lý giao diện sau:

3.4.1. Cơ chế Chuyển động (Animation)

- JavaScript là ngôn ngữ đơn luồng, nếu chạy vòng lặp thuật toán thông thường, kết quả sẽ hiện ra ngay lập tức. Để tạo hiệu ứng "mô phỏng", đề tài sử dụng hàm sleep(ms) kết hợp với từ khóa await trong hàm async.
- Độ trễ:
 - Tốc độ duyệt đỉnh: 30ms/bước (đủ nhanh để không chờ lâu, đủ chậm để mắt thường quan sát sự lan truyền).
 - Tốc độ vẽ đường đi (Path reconstruction): 150ms/bước (nhấn mạnh kết quả cuối cùng).

3.4.2. Hiệu ứng màu sắc

Các trạng thái của ô được quy định rõ trong style.css:

- .start (Xanh lá): Điểm bắt đầu.
- .end (Đỏ): Điểm kết thúc.

- `.wall` (Xám đen): Chướng ngại vật.
- `.visited-bfs` (Xanh dương sáng): Vùng BFS đã quét qua (hiệu ứng loang).
- `.visited-dfs` (Cam): Vùng DFS đã quét qua (hiệu ứng đường dẫn).
- `.path` (Vàng rực): Đường đi tìm được từ Start đến End.

3.4.3. Tương tác người dùng

- Kéo thả (Drag & Drop): Cho phép thay đổi vị trí Start/End node động ngay trên lưới.
- Nhập liệu Ma trận: Chức năng `applyMatrix()` phân tích chuỗi văn bản đầu vào, tự động điều chỉnh kích thước lưới (rows, cols) và bố trí tường theo dữ liệu số.

CHƯƠNG 4: KẾT QUẢ NGHIÊN CỨU

4.1 Tổng quan kết quả đạt được

Sau quá trình thiết kế, lập trình và kiểm thử, ứng dụng mô phỏng giải thuật tìm kiếm trên đồ thị đã được xây dựng thành công. Giao diện trực quan, dễ sử dụng, cho phép người dùng tương tác trực tiếp với lưới đồ thị, vẽ tường, đặt điểm bắt đầu/kết thúc, nhập ma trận số và quan sát quá trình duyệt của thuật toán BFS và DFS theo thời gian thực.

Ứng dụng hỗ trợ mô phỏng riêng lẻ từng thuật toán hoặc chạy song song để so sánh trực quan. Các hiệu ứng màu sắc, chuyển động và khả năng tương tác giúp người học dễ dàng hình dung cách thuật toán hoạt động, từ đó hiểu sâu hơn về bản chất và sự khác biệt giữa BFS và DFS.

4.2. Kết quả mô phỏng

4.2.1. Giao diện ứng dụng

Khi khởi động chương trình, giao diện mặc định hiển thị một lưới ô vuông kích thước 20x20.

- **Bảng điều khiển (Bên trái):**

- Hiển thị đầy đủ các nút chức năng:

- + Xóa/ Tạo mới

- + Chạy BFS, DFS

- + So sánh cả hai

- Bảng chú thích màu sắc.

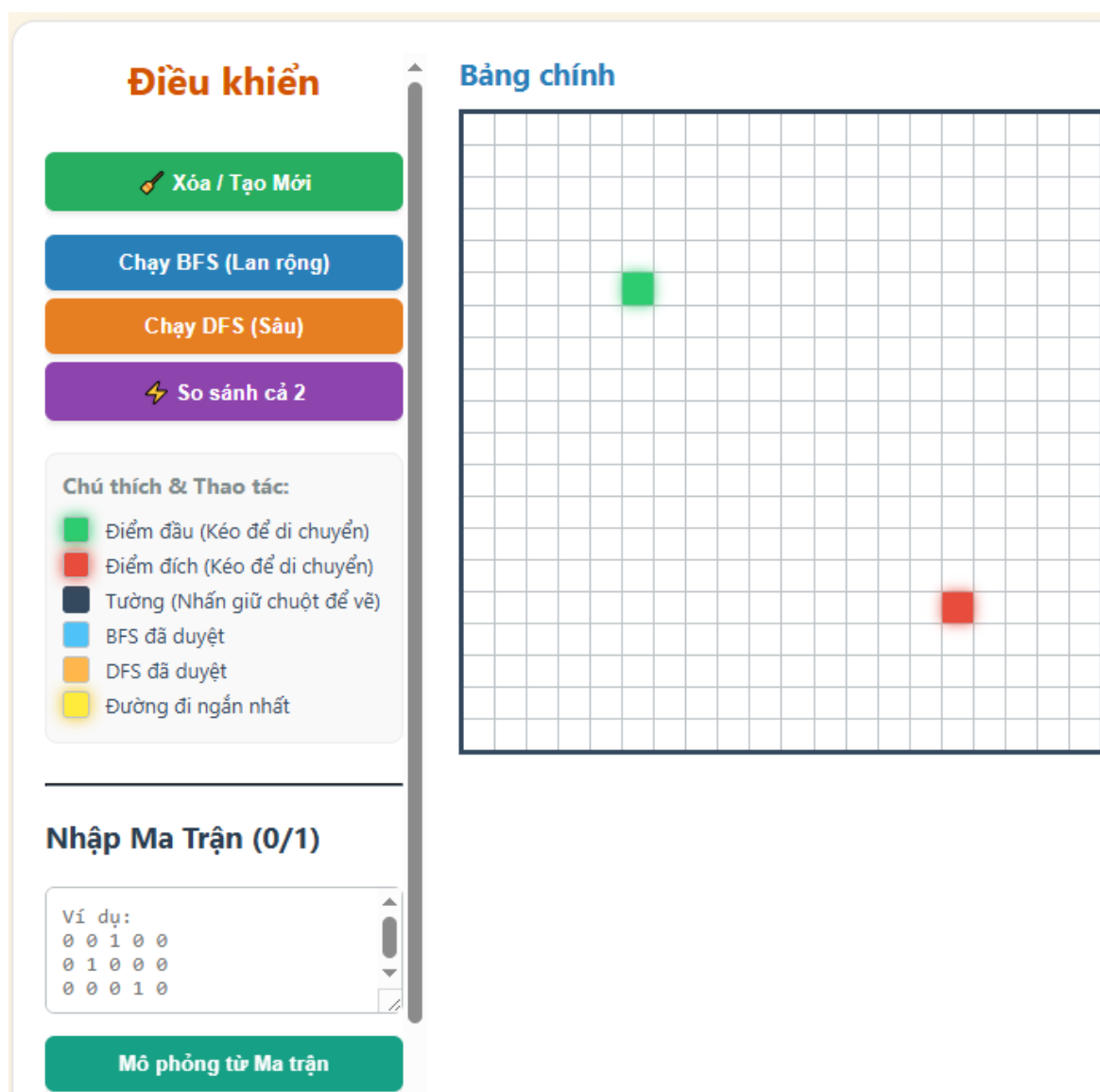
- Ô nhập ma trận

- **Lưới mô phỏng (Bên phải):**

- Các ô vuông trắng đại diện cho đường đi

- Điểm bắt đầu (Start) màu xanh lá

- Điểm kết thúc (End) màu đỏ được đặt mặc định tại vị trí (5,5) và (15,15).



Hình 6 Giao diện chính

4.2.2. Mô phỏng thuật toán BFS/DFS

4.2.2.1 Thuật toán BFS

- BFS lan rộng theo từng lớp, các ô được duyệt theo thứ tự khoảng cách từ điểm bắt đầu.
- Màu xanh dương thể hiện vùng đã duyệt, màu vàng thể hiện đường đi ngắn nhất từ điểm đầu đến điểm đích.
- Khi gặp tường (ô màu xám) hoặc biên lưới, thuật toán tự động bỏ qua và tiếp tục mở rộng sang các ô hợp lệ.

- Kết quả cho thấy BFS luôn tìm được đường đi tối ưu nếu tồn tại, bất kể vị trí tường hay độ phức tạp của bản đồ.

Kiểm tra với Ma trận 5x5:

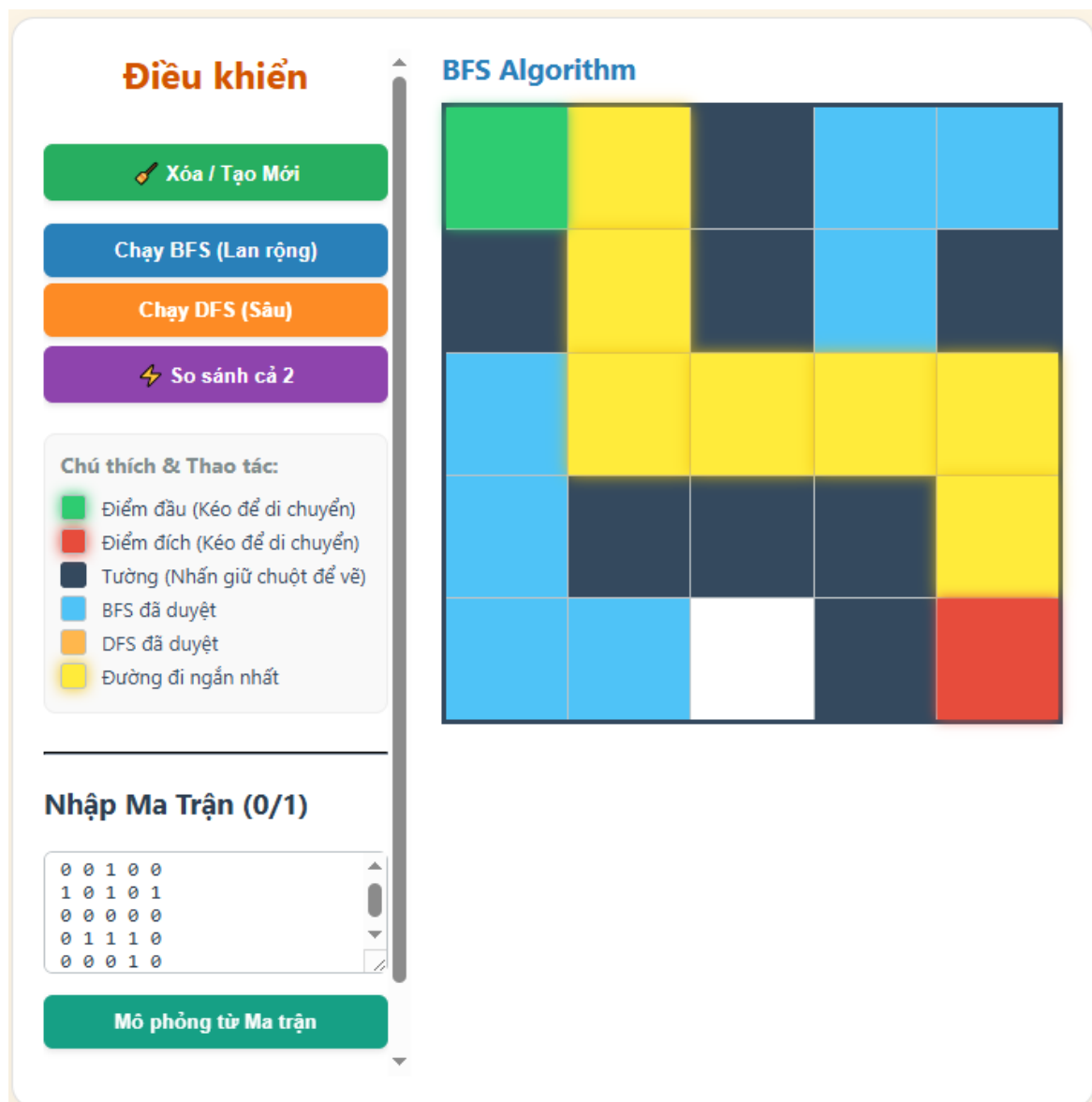
0	0	1	0	0
1	0	1	0	1
0	0	0	0	0
0	1	1	1	0
0	0	0	1	0

Kết quả



Hình 7 Nhập ma trận

Kết quả chạy BFS:



Hình 8 Kết quả mô phỏng thuật toán BFS

4.2.2.2. Thuật toán DFS

- DFS đi sâu theo một nhánh, thể hiện rõ quá trình “đi sâu rồi quay lui”.
- Màu cam thể hiện vùng đã duyệt, đường đi được vẽ bằng màu vàng sau khi tìm thấy điểm đích.
- Đường đi tìm được có thể không ngắn nhất, nhưng phản ánh đúng cơ chế duyệt của DFS.

- Thứ tự duyệt phụ thuộc vào cách sắp xếp hướng đi (ưu tiên lên, xuống, trái, phải), giúp người học thấy rõ ảnh hưởng của cấu trúc thuật toán.

Kiểm tra với Ma trận 5x5:

0	0	1	0	0
1	0	1	0	1
0	0	0	0	0
0	1	1	1	0
0	0	0	1	0

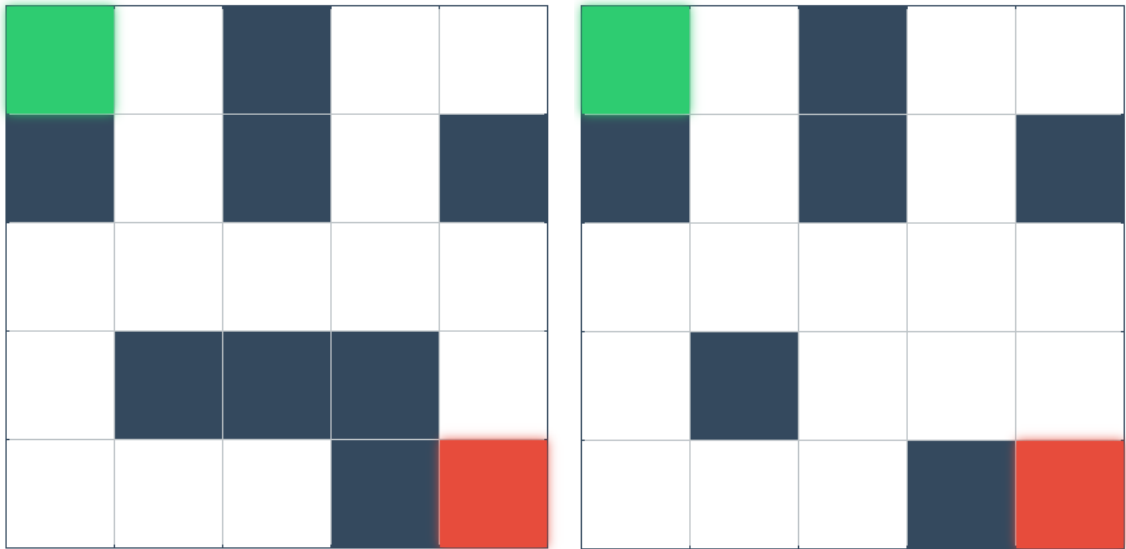
Kết quả



Hình 9 Kết quả mô phỏng thuật toán DFS

4.2.3. Thử nghiệm tương tác

Thay đổi vị trí chương ngại vật



Hình 10 Kết quả thay đổi vị trí chương ngại vật

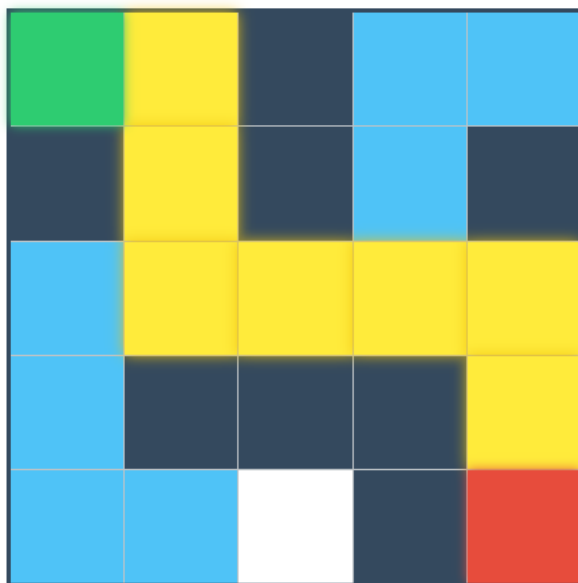
Kết quả duyệt BFS:

BFS:

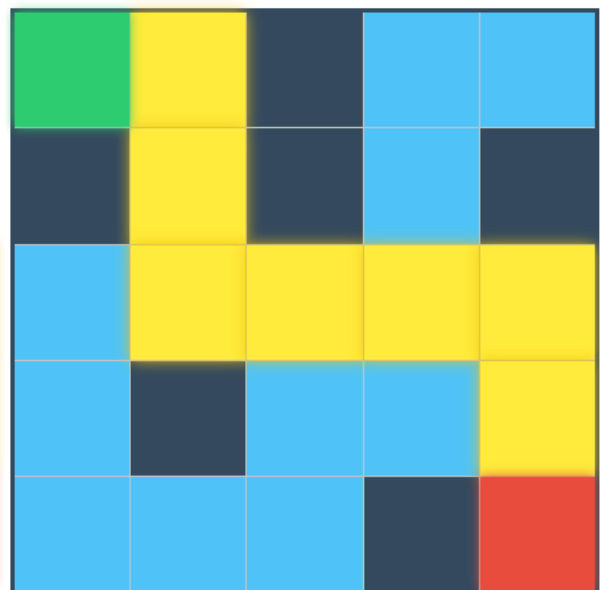
Trước

Sau

BFS Algorithm



BFS Algorithm



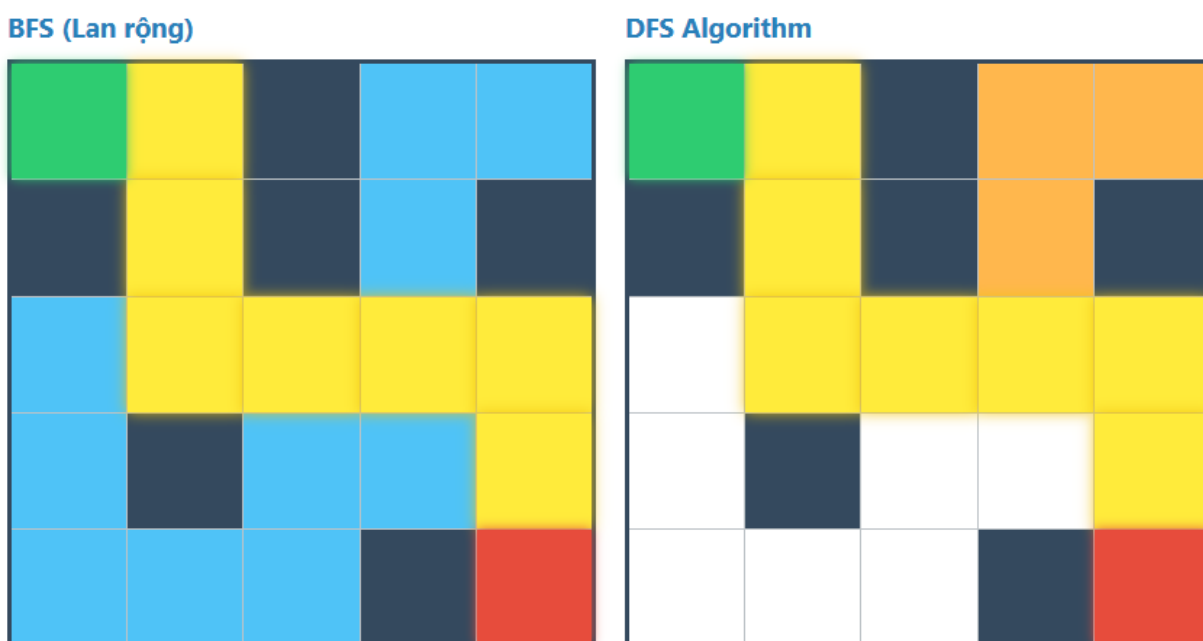
Hình 11 Mô phỏng chạy BFS sau khi thay đổi vị trí chương ngại vật

4.2.4. Chức năng duyệt song song

Đây là tính năng nâng cao giúp so sánh trực quan hiệu suất.

- **Hiển thị:** Màn hình chia làm hai lưới riêng biệt. Lưới trên chạy BFS, lưới dưới chạy DFS.
- **Quan sát:** BFS lan rộng đều, DFS đi sâu theo nhánh.
 - BFS mất nhiều bước duyệt hơn nhưng cho kết quả tối ưu.
 - DFS duyệt nhanh hơn nhưng đường đi có thể vòng vèo.

Giao diện chia đôi lưới (grid1 và grid2) giúp minh họa trực quan, tăng hiệu quả giảng dạy và học tập.



Hình 12 Chức năng duyệt song song

4.3. Nhận xét chung

Tổng quan

- Tính trực quan: hiệu ứng màu sắc rõ ràng, dễ nhận biết các trạng thái duyệt.
- Tính tương tác: người dùng có thể kéo thả điểm bắt đầu/kết thúc, vẽ tường, nhập ma trận.
- Tính ổn định: ứng dụng hoạt động mượt mà, không phát sinh lỗi trong quá trình kiểm thử.

- Tính giáo dục: hỗ trợ sinh viên hiểu sâu hơn về thuật toán, phù hợp với chương trình học cơ sở ngành.
- Tính mở rộng: có thể tích hợp thêm các thuật toán nâng cao như Dijkstra, A* trong tương lai.

Ưu điểm:

- Giao diện tương tác cao (Interactive UI): Khác với các mô hình tĩnh trong sách giáo khoa, ứng dụng cho phép người dùng tự tay thiết kế mê cung thông qua thao tác vẽ chuột và kéo thả. Điều này giúp tăng hứng thú học tập và khả năng tư duy về không gian.
- Mô phỏng thời gian thực: Việc sử dụng kỹ thuật `async/await` để tạo độ trễ giúp mắt thường quan sát được từng bước "loang" của thuật toán. Hệ thống màu sắc (Color Coding) phân biệt rõ ràng giữa các trạng thái: Chưa duyệt - Đang duyệt - Đã duyệt - Đường đi.
- Khả năng tùy biến dữ liệu: Việc tích hợp tính năng nhập dữ liệu từ văn bản (Matrix Input) giúp ứng dụng không bị giới hạn trong việc vẽ tay, mà có thể xử lý các bài toán có cấu trúc dữ liệu định sẵn.
- Nền tảng công nghệ hiện đại: Ứng dụng được đóng gói bằng ElectronJS, giúp cài đặt và chạy độc lập trên Windows như một phần mềm chuyên nghiệp, không phụ thuộc vào trình duyệt web.

Hạn chế

Bên cạnh những kết quả đạt được, ứng dụng vẫn còn một số hạn chế nhỏ:

- Chưa hỗ trợ trọng số: Hiện tại ứng dụng chỉ mô phỏng trên đồ thị không trọng số (Unweighted Graph), do đó chưa thể so sánh với các thuật toán tối ưu hơn như Dijkstra hay A* trong các bài toán giao thông thực tế.
- Kích thước lưới hiển thị: Kích thước hiển thị lưới phụ thuộc vào độ phân giải màn hình. Với các ma trận nhập vào quá lớn (ví dụ $> 50 \times 50$), các ô sẽ bị thu nhỏ lại gây khó quan sát.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết quả đạt được

Sau quá trình nghiên cứu và hiện thực hóa, đề tài “Xây dựng ứng dụng mô phỏng giải thuật tìm kiếm trên đồ thị” đã hoàn thành với các kết quả nổi bật sau:

- Ứng dụng mô phỏng BFS và DFS hoạt động ổn định, trực quan, dễ sử dụng.
- Giao diện thân thiện, hỗ trợ tương tác trực tiếp với lưới đồ thị, cho phép người dùng vẽ tường, đặt điểm bắt đầu/kết thúc, nhập ma trận số.
- Hiệu ứng màu sắc và chuyển động giúp minh họa rõ ràng quá trình duyệt của từng thuật toán.
- Chức năng so sánh song song BFS và DFS giúp người học dễ dàng nhận biết sự khác biệt về cách duyệt và hiệu quả tìm đường.

5.2. Ưu nhược điểm

Ưu điểm

- Giao diện trực quan, sinh động, tạo hứng thú học tập.
- Ứng dụng hoạt động mượt mà, dễ triển khai trên nhiều hệ điều hành.
- Có khả năng mở rộng thêm thuật toán khác trong tương lai.
- Phù hợp với mục tiêu giảng dạy và học tập môn Lý thuyết đồ thị.

Nhược điểm

- Mới chỉ hỗ trợ hai thuật toán cơ bản (BFS và DFS).
- Giao diện còn đơn giản, chưa có chức năng lưu trạng thái hoặc xuất kết quả.
- Hiệu năng có thể giảm khi mô phỏng trên lưới lớn hoặc dữ liệu phức tạp.

5.3. Hướng phát triển

Trong tương lai, ứng dụng có thể được nâng cấp theo các hướng sau:

- Tích hợp thêm các thuật toán nâng cao như Dijkstra, A*, Bellman-Ford.
- Bổ sung chức năng lưu trạng thái, xuất kết quả ra file hoặc chia sẻ qua mạng.
- Tối ưu hiệu năng xử lý và hiển thị trên lưới lớn.

- Phát triển phiên bản web trực tuyến hoặc ứng dụng di động để mở rộng đối tượng sử dụng.
- Tăng cường khả năng tùy chỉnh giao diện và tốc độ mô phỏng theo nhu cầu người dùng.

5.4. Kết luận

Đề tài đã hoàn thành đúng mục tiêu đề ra: xây dựng một công cụ mô phỏng trực quan giúp sinh viên hiểu rõ hơn về cách hoạt động của các giải thuật tìm kiếm trên đồ thị. Việc kết hợp giữa lý thuyết và thực nghiệm, giữa thuật toán và giao diện trực quan, đã tạo nên một sản phẩm có giá trị ứng dụng cao trong giảng dạy và học tập.

TÀI LIỆU THAM KHẢO

- [1] Đỗ Xuân Lôi, *Cấu trúc dữ liệu và Giải thuật*, Nhà xuất bản Đại học Quốc gia Hà Nội, 2007.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.
- [3] Mozilla Developer Network (MDN), "JavaScript Guide - Algorithms and Data Structures,". <https://developer.mozilla.org/en-US/docs/Web/JavaScript>.
- [4] OpenJS Foundation, "Electron Documentation,". <https://www.electronjs.org/docs/latest/>.
- [5] Node.js Foundation, "Node.js v20.x Documentation," <https://nodejs.org/docs/latest/api/>.